

Projektdokumentation

Automatisierung der Holzverarbeitung

Digitale Mengenerfassung am Förderband mit dem Raspberry Pi 4

Projekt Sommer 2026

Umschulung Fachinformatiker - Fachrichtung Anwendungsentwicklung

Projektbeauftragter:	Mark Ohler
Ausbildungsbetrieb:	BFW Schömberg
Durchführungszeitraum:	08.06. - 03.07.2026
Abgabedatum:	02.07.2026

Inhaltsverzeichnis

1	Einleitung	4
1.1	Ausgangssituation	4
1.2	Projektziel.....	4
1.3	Anforderungen an das System	4
2	Projektplanung	5
2.1	Projektphasen.....	5
2.2	Entwicklungsumgebung und Programmiersprache	7
2.3	Eingesetzte Hardware	7
3	Durchführung des Projekts.....	8
3.1	Einrichtung der Arbeitsumgebung.....	8
3.2	Auswahl und Aufbau der Sensorik.....	9
3.2.1	Anforderungen an die Erkennung.....	9
3.2.2	Fehlversuch 1: Sichtbarer Laser an einem Infrarot-Empfänger.....	9
3.2.3	Fehlversuch 2: Der Infrarot-Empfänger braucht ein getaktetes Signal.....	9
3.2.4	Fehlversuch 3: Der Abstandssensor löst unkontrolliert aus	10
3.2.5	Die Lösung, die funktioniert hat: Laser und Lichtsensor-Modul.....	10
3.2.6	Übersicht der getesteten Sensoren	11
3.3	Aufbau und Verdrahtung.....	11
3.4	Programmierung der Zählsoftware.....	12
3.4.1	Aufbau des Programms.....	12
3.4.2	Zählen über den Signalwechsel	12
3.4.3	Entprellung gegen Mehrfachzählung	13
3.4.4	Fehlzählung beim Start vermeiden	13
3.5	Anzeige, Chargen und Speicherung	13
3.5.1	Anzeige auf dem Display.....	13
3.5.2	Chargen über das Taster-Modul.....	13
3.5.3	Speicherung in einer XML-Datei.....	13
3.5.4	Automatischer Tageswechsel um Mitternacht	14
3.5.5	Schutz vor Datenverlust bei Stromausfall.....	14
3.6	Bedienung der Anlage	14
4	Test und Qualitätssicherung.....	15
4.1	Vorgehen beim Testen	15
4.2	Testfälle und Ergebnisse	15
4.3	Besondere Prüfungen.....	15
5	Projektabschluss.....	16
5.1	Soll-Ist-Vergleich	16
5.2	Abweichungen zur Projektskizze	17
6	Fazit und Ausblick.....	18

Tabellenverzeichnis

Tabelle 1: Abkürzungen	3
Tabelle 2: Zeitplan	6
Tabelle 3: Entwicklungsumgebung und Programmiersprache	7
Tabelle 4: Eingesetzte Hardware	7
Tabelle 5: Übersicht der getesteten Sensoren	11
Tabelle 6: Pin-Anschlüsse.....	12
Tabelle 7: Testfälle und Ergebnisse	15
Tabelle 8: Soll-Ist-Vergleich	16

Abbildungsverzeichnis

Abbildung 1: Zeitlicher Ablauf der Projektphasen (Gantt-Diagramm).....	5
Abbildung 2: Aufbau und Signalfuss des Stückzählers	11

Abkürzungen

Abkürzung	Bedeutung
CSV	Comma-Separated Values - einfaches Tabellenformat für Daten
GPIO	General Purpose Input/Output - die Ein- und Ausgänge des Raspberry Pi
I ² C	Inter-Integrated Circuit - Datenbus zur Anbindung des Displays
LCD	Liquid Crystal Display - Flüssigkristallanzeige
LDR	Light Dependent Resistor - lichtabhängiger Widerstand
OS	Operating System - Betriebssystem
RTC	Real Time Clock - Echtzeituhr, die die Uhrzeit auch ohne Strom weiterführt
SSH	Secure Shell - verschlüsselter Fernzugriff auf den Raspberry Pi
XML	Extensible Markup Language - Format zur strukturierten Speicherung

Tabelle 1: Abkürzungen

1 Einleitung

1.1 Ausgangssituation

Im Sägewerk werden die fertig zugeschnittenen Holzbretter nach der letzten Bearbeitungsstation ins Lager gebracht. Bisher passiert das von Hand, und wie viele Bretter durchgelaufen sind, wird mit Zettel und Stift notiert. Das kostet Zeit, und es schleichen sich leicht Fehler ein. Wenn man später wissen will, wie viel an einem Tag oder in einem Monat produziert wurde, muss man die Zettel mühsam zusammenrechnen. Eine saubere und schnelle Auswertung ist auf diese Weise kaum möglich.

Gesucht ist deshalb eine Lösung, die das Zählen übernimmt, den Stand jederzeit anzeigt und die Zahlen automatisch für spätere Auswertungen festhält.

1.2 Projektziel

Ziel ist ein System, das die Bretter automatisch zählt, sobald sie das Band verlassen. Eine Lichtschranke erkennt jedes Brett, ein Raspberry Pi zählt mit, zeigt den aktuellen Stand auf einem kleinen Display an und speichert die Zahlen, damit man sie später auswerten kann.

1.3 Anforderungen an das System

Bevor es an die Umsetzung ging, wurde festgelegt, was das System können muss. Daraus ergaben sich die folgenden Anforderungen.

Muss-Anforderungen:

- Jedes durchlaufende Brett wird sicher erkannt und genau einmal gezählt.
- Der aktuelle Zählerstand ist jederzeit auf einem Display ablesbar.
- Die Zählung läuft berührungslos, ohne dass die Bretter angefasst werden müssen.
- Die Daten werden dauerhaft gespeichert und bleiben auch nach einem Neustart erhalten.

2 Projektplanung

2.1 Projektphasen

Das Projekt war auf 40 Stunden ausgelegt und in fünf Phasen aufgeteilt. Zuerst wurden die Anforderungen geklärt und geplant, danach die Hardware aufgebaut und verdrahtet. Den größten Block bildete die Programmierung. Zum Schluss folgten die Tests und die Dokumentation samt Vorführung. Die folgende Abbildung zeigt, wie die Phasen zeitlich aufeinander folgen.

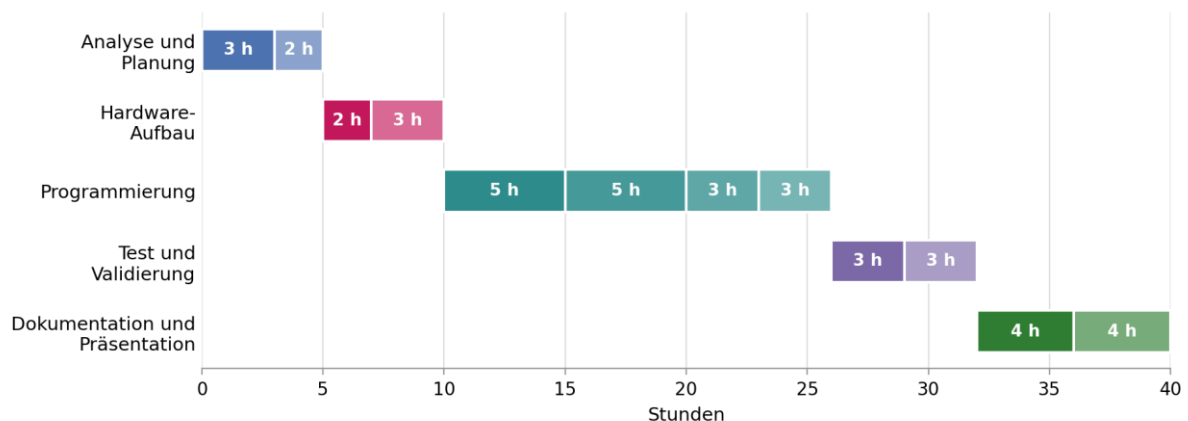


Abbildung 1: Zeitlicher Ablauf der Projektphasen (Gantt-Diagramm)

Die Abbildung macht deutlich, wie sich der Gesamtaufwand von 40 Stunden über die fünf Projektphasen verteilt und wie diese zeitlich aufeinander aufbauen. Jede Phase ist zusätzlich in einzelne Arbeitsschritte von maximal fünf Stunden unterteilt, erkennbar an den Farbabstufungen innerhalb der jeweiligen Phase. Diese Feingliederung erleichtert es, den Fortschritt während der Projektwoche zu überprüfen und die einzelnen Aufgaben realistisch einzuplanen. Erwartungsgemäß nimmt die Programmierung mit 16 Stunden den größten Anteil ein, da hier die Kernfunktionalität der Zählsoftware entsteht.

Die folgende Übersicht fasst die Phasen in Arbeitsschritte von maximal 5 Stunden unterteilt, mit ihrem geplanten Aufwand zusammen:

Phase	Inhalt	Geplant
Analyse und Planung		5 h
	Ist-Aufnahme des Produktionsprozesses: Messpunkte festlegen, Anforderungen abstimmen	3 h
	Schaltplan erstellen, Komponentenliste finalisieren, Projektstruktur definieren	2 h
Hardware-Aufbau		5 h
	Raspberry Pi einrichten: Betriebssystem installieren, SSH-Zugang konfigurieren, VS Code Remote verbinden	2 h
	Lichtschrankensensor und LCD-Display (I ² C) gemäß Schaltplan verdrahten und Funktion überprüfen	3 h
Programmierung		16 h
	GPIO-Eingang konfigurieren, Sensor-Signal auslesen, Entprellung implementieren	5 h
	Zählerlogik entwickeln: Inkrementierung bei jedem Signalimpuls, Reset- und Startfunktion	5 h
	LCD-Ausgabe implementieren: aktuellen Zählerstand nach jedem Brett live anzeigen	3 h
	Speicherung mit Zeitstempel realisieren, Absicherung gegen Fehlzählung integrieren	3 h
Test und Validierung		6 h
	Funktionstest im Normalbetrieb: Zählgenauigkeit und Anzeige verifizieren	3 h
	Fehlertests: Doppelimpuls und Sonderfälle simulieren, Grenzwerte und Startverhalten prüfen	3 h
Dokumentation und Präsentation		8 h
	Technische Dokumentation erstellen: Schaltplan, Code-Kommentare, Bedienungshinweise	4 h
	Prototyp aufbauen und Funktionsweise des Gesamtsystems demonstrieren	4 h

Tabelle 2: Zeitplan

2.2 Entwicklungsumgebung und Programmiersprache

Programmiert wurde nicht direkt am Raspberry Pi, sondern von einem Windows-PC aus. Dafür wird der Editor Visual Studio Code über Remote-SSH mit dem Pi verbunden. Ein eigener Monitor am Pi ist damit nicht nötig.

Als Sprache kommt Python 3 zum Einsatz. Python ist die Standardsprache auf dem Raspberry Pi, es gibt fertige Bibliotheken für die Anschlüsse (GPIO) und für das Display, und damit lassen sich alle Aufgaben gut lösen, vom Auslesen des Sensors bis zum Speichern der Daten.

Bereich	Auswahl
Zielgerät	Raspberry Pi 4 mit Raspberry Pi OS (Lite-Variante, ohne Oberfläche)
Entwicklungs-PC	Windows-PC
Editor	Visual Studio Code mit Remote-SSH
Programmiersprache	Python 3
Bausteine (Bibliotheken)	RPi.GPIO für die Anschlüsse, RPLCD für das Display

Tabelle 3: Entwicklungsumgebung und Programmiersprache

2.3 Eingesetzte Hardware

Für den Aufbau wurden günstige, frei verfügbare Bauteile verwendet. Das Herzstück ist der Raspberry Pi, der den Sensor ausliest, zählt, das Display ansteuert und die Daten speichert. Die folgende Tabelle gibt einen Überblick über die verwendeten Teile.

Bauteil	Bezeichnung	Aufgabe
Recheneinheit	Raspberry Pi 4 (4 GB)	zählt, zeigt an und speichert die Daten
Laser-Modul	KY-008	Sender der Lichtschranke
Lichtsensormodul	KY-018	Empfänger der Lichtschranke
Taster-Modul	KY-004	startet eine neue Charge
Anzeige	LCD-Display 16×2 (I ² C)	zeigt Zählerstand und Status
Echtzeituhr	DS1307 (I ² C)	liefert die richtige Uhrzeit, auch ohne Internet
Zubehör	MicroSD-Karte, Netzteil, Steckbrett, Kabel	Betriebssystem, Strom und Verdrahtung
Entwicklungs-PC	Windows-PC mit VS-Code	Programmierung über Remote-SSH

Tabelle 4: Eingesetzte Hardware

3 Durchführung des Projekts

Die Umsetzung lief in den geplanten Phasen ab. Den meisten Aufwand machte die Auswahl des Sensors, denn der erste Ansatz funktionierte nicht und es mussten mehrere Varianten ausprobiert werden. Sobald der passende Sensor gefunden war, ging der Rest deutlich schneller von der Hand.

3.1 Einrichtung der Arbeitsumgebung

Auf den Raspberry Pi wurde Raspberry Pi OS (Lite) installiert, also ohne grafische Oberfläche

Der Zugriff vom PC auf den Pi läuft über SSH. Statt sich jedes Mal mit Passwort anzumelden, wurde ein Schlüssel-Verfahren eingerichtet: Ein Teil des Schlüssels liegt auf dem Pi, der andere auf dem PC. Nur wenn beide zusammenpassen, kommt man rein. Nachdem das einmal getestet war, wurde die Passwort-Anmeldung am Pi ganz abgeschaltet, sodass nur noch der hinterlegte Schlüssel funktioniert. Das macht den Zugang sicherer. Über VS Code mit Remote-SSH wurde dann die ganze Zeit direkt auf dem Pi gearbeitet.

Zum Schluss wurde noch die Echtzeituhr (DS1307) im Betriebssystem aktiviert. Sie wird einmalig in der Konfiguration des Pi eingetragen. Danach holt sich der Pi bei jedem Start die richtige Uhrzeit von der Uhr, auch ohne Internet. Das ist wichtig, weil der Mitternachts-Reset und die Zeitstempel in den Daten auf die korrekte Uhrzeit angewiesen sind. Ohne diese Uhr würde der Pi nach einem Stromausfall ohne Netzwerk mit einer falschen Zeit starten.

3.2 Auswahl und Aufbau der Sensorik

3.2.1 Anforderungen an die Erkennung

Die Lichtschranke muss jedes Brett sicher erkennen, und zwar genau einmal. Dafür braucht es ein klares An/Aus-Signal, genug Reichweite über die Breite des Bands und einen Betrieb, der dauerhaft stabil läuft. Genau die Auswahl des passenden Sensors war der Teil, der am meisten Zeit gekostet hat. Mehrere Sensoren wurden ausprobiert, bis einer wirklich zuverlässig lief. Die drei wichtigsten Fehlversuche und wie damit umgegangen wurde, stehen in den nächsten Abschnitten.

3.2.2 Fehlversuch 1: Sichtbarer Laser an einem Infrarot-Empfänger

Der erste Aufbau bestand aus einem sichtbaren roten Laser als Sender und einem Infrarot-Empfänger als Gegenstück. Der Empfänger reagierte auf den Laser und erkannte den Strahl. Auf den ersten Blick sah es also so aus, als würde es funktionieren. Brauchbar war das Signal aber trotzdem nicht.

Warum es nicht ging: Der Strahl wurde zwar erkannt, aber nur ganz kurz. Statt durchgehend zu melden, dass Licht ankommt, reagierte der Empfänger nur in dem Moment, in dem sich das Licht änderte, und kippte danach wieder zurück. Für eine Lichtschranke, die jederzeit wissen muss, ob der Strahl gerade frei oder unterbrochen ist, war das nicht zu gebrauchen.

Lösung: Damit war klar, dass der Empfänger genauer untersucht werden musste. Das führte direkt zum nächsten Schritt, in dem sich zeigte, woran das eigentlich liegt.

3.2.3 Fehlversuch 2: Der Infrarot-Empfänger braucht ein getaktetes Signal

Um die Ursache zu finden, wurde der Empfänger mit einer Infrarot-Sendediode (KY-005) getestet, also mit genau der Lichtart, für die er gedacht ist. Doch auch damit lieferte er kein brauchbares Dauersignal.

Warum es nicht ging: Dieser Empfänger ist eigentlich für Fernbedienungen gebaut. Er erwartet kein gleichmäßiges Dauerlicht, sondern ein sehr schnell an- und ausgeschaltetes Signal von 38 kHz. Ein Sender, der einfach nur durchgehend leuchtet, wird von ihm sogar absichtlich ausgeblendet.

Es wurde versucht, dieses schnelle Takten auf dem Raspberry Pi nachzubauen. Die einfache Software-Lösung schaffte aber nur rund 10 kHz und damit zu wenig. Über eine hardwarenahe Einstellung ließen sich die 38 kHz zwar erzeugen, doch der Empfänger blendete das gleichmäßige Signal trotzdem wieder aus.

Was dabei klar wurde: Dieser Empfänger-Typ passt einfach nicht für eine Lichtschranke, die dauerhaft überwacht. Deshalb wurde dieser Weg nicht weiter verfolgt.

3.2.4 Fehlversuch 3: Der Abstandssensor löst unkontrolliert aus

Als Nächstes wurde ein Infrarot-Abstandssensor (KY-032) getestet. Im ersten Moment sah es gut aus: Der Sensor reagierte, das Signal wechselte, und es wirkte so, als würde er die Bretter zuverlässig erkennen. Bei genauerem Hinsehen zeigte sich aber, dass auch hier das Signal flackerte und der Sensor nicht zuverlässig genau dann auslöste, wenn der Strahl tatsächlich unterbrochen wurde.

Warum es nicht ging: Der Sensor arbeitet nicht wie eine echte Lichtschranke, sondern misst die Entfernung zu allem, was sich vor ihm befindet. Er reagierte deshalb nicht gezielt auf das Brett im Strahlbereich, sondern allgemein auf Bewegung in seiner Nähe, etwa wenn eine Hand zu nah am Sensor vorbeikam oder sich im Hintergrund etwas bewegte. Dadurch löste er auch dann aus, wenn gar kein Brett durch die Lichtschranke lief oder löste nicht aus obwohl ein Brett den Bereich passierte. Für eine Zählung, die genau und nur bei einem durchlaufenden Brett auslösen soll, war das nicht brauchbar.

Was dabei klar wurde: Ein Abstandssensor misst Entfernung und Bewegung allgemein, das ist etwas anderes als eine echte Lichtschranke, die nur auf die Unterbrechung eines gezielten Strahls reagiert. Für die geforderte genaue, eindeutige Erkennung war dieser Sensor-Typ deshalb ungeeignet.

3.2.5 Die Lösung, die funktioniert hat: Laser und Lichtsensor-Modul

Die endgültige Lösung besteht aus einem Laser-Modul (KY-008) als Sender und einem Lichtsensor-Modul (KY-018) als Empfänger. Diese Kombination lief von Anfang an stabil, und damit war das größte Problem des Projekts gelöst.

So funktioniert es: Der Laser leuchtet dauerhaft auf das Lichtsensor-Modul. Auf dem Modul sitzt ein kleiner Vergleichs-Chip (LM393), der die gemessene Helligkeit mit einem voreingestellten Wert vergleicht. Trifft der Laser auf den Sensor, gibt das Modul den einen Schaltzustand aus. Schiebt sich ein Brett dazwischen und unterbricht den Strahl, kippt es in den anderen. So entsteht ein sauberes An/Aus-Signal, das der Raspberry Pi ohne Umwege direkt verarbeiten kann.

Diese Lösung hat mehrere Vorteile:

- Der gebündelte Laserstrahl ist stark und gut sichtbar und lässt sich genau über die Breite des Bands ausrichten.
- Das Modul liefert von sich aus ein klares An/Aus-Signal. Es braucht kein getaktetes Signal und keine zusätzlichen Schaltungen.
- Die Bauteile sind günstig und leicht zu beschaffen.

Die folgende Abbildung zeigt, wie die Teile zusammenarbeiten: Der Laser strahlt durch den Bereich, in dem die Bretter durchlaufen, auf das Lichtsensor-Modul. Dieses meldet jede Unterbrechung an den Raspberry Pi, der zählt, das Display ansteuert und die Daten speichert. Über das Taster-Modul lässt sich eine neue Charge starten. Über das Taster-Modul lässt sich eine neue Charge starten.

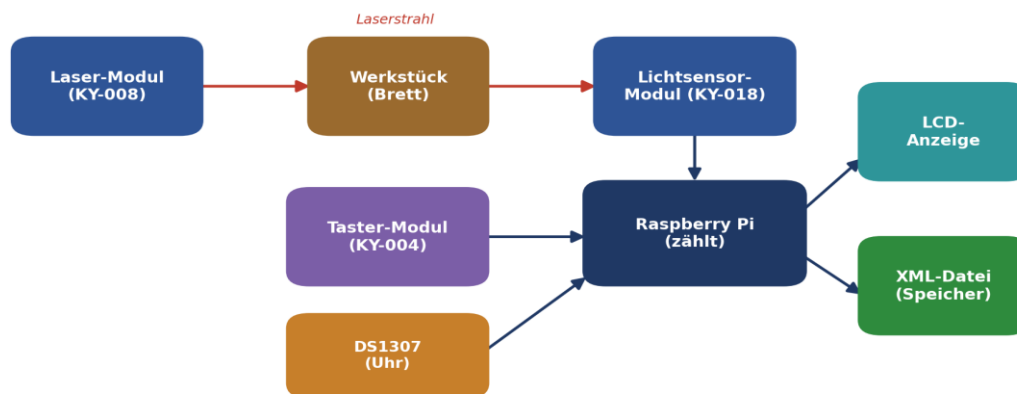


Abbildung 2: Aufbau und Signalfluss des Stückzählers

3.2.6 Übersicht der getesteten Sensoren

Zur besseren Übersicht fasst die folgende Tabelle zusammen, welche Sensoren ausprobiert wurden, woran es jeweils gehakt hat und wie die Entscheidung am Ende ausfiel.

Sensor	Problem	Ergebnis
Sichtbarer Laser + Infrarot-Empfänger	Empfänger reagiert nur auf Änderungen, nicht auf Dauerlicht	verworfen
Infrarot-Sendediode + Infrarot-Empfänger	Empfänger braucht getaktetes Signal, blendet Dauerlicht aus	verworfen
Infrarot-Abstandssensor (KY-032)	reagiert auf allgemeine Bewegung statt gezielt auf die Lichtschranke, Signal flackert	verworfen
Laser-Modul (KY-008) + Lichtsensor-Modul (KY-018)	läuft stabil, klares An/Aus-Signal, leicht auszurichten	gewählt

Tabelle 5: Übersicht der getesteten Sensoren

3.3 Aufbau und Verdrahtung

Aufgebaut wurde alles auf einem Steckbrett. Das hat den Vorteil, dass sich einzelne Teile schnell umstecken lassen, ohne löten zu müssen. Gerade während der Sensor-Suche war das sehr praktisch. Der Laser und das Lichtsensor-Modul stehen sich gegenüber, sodass der Strahl genau auf den Sensor trifft. Dazwischen ist Platz, durch den die Bretter laufen.

Versorgt werden die Module mit 3,3 Volt vom Raspberry Pi. Das hat einen wichtigen Grund: Das Ausgangssignal des Sensors liegt damit von vornherein im sicheren Bereich für die Eingänge des Pi. Würde man die Module mit 5 Volt versorgen, könnte am Ausgang eine zu hohe Spannung anliegen, die den Pi beschädigt. Mit 3,3 Volt ist man auf der sicheren Seite.

Das Display hängt am sogenannten I²C-Bus. Dafür reichen zwei Datenleitungen (SDA und SCL) sowie Strom und Masse. Über diesen Bus spricht der Pi das Display mit einer festen Adresse (0x27) an. Der Taster ist so angeschlossen, dass am Eingang des Pi immer ein eindeutiger Pegel anliegt; dafür sorgt ein interner Widerstand, der per Software eingeschaltet wird. So wird ein Tastendruck zuverlässig erkannt.

Welche Bauteile an welchem Anschluss des Raspberry Pi hängen, zeigt die folgende Tabelle. Die Nummern sind die GPIO-Nummern, mit denen die Anschlüsse im Programm angesprochen werden.

Anschluss am Pi	Bauteil	Aufgabe
GPIO 18	Laser-Modul (KY-008)	schaltet den Laser ein
GPIO 25	Lichtsensormodul (KY-018)	liest das An/Aus-Signal ein
GPIO 17	Taster-Modul (KY-004)	erkennt den Tastendruck
SDA / SCL (I ² C, 0x27)	LCD-Display, Uhr	steuert die Anzeige an, liest Uhrzeit ein

Tabelle 6: Pin-Anschlüsse

3.4 Programmierung der Zählsoftware

3.4.1 Aufbau des Programms

Das Programm ist in Python geschrieben und läuft direkt auf dem Pi. Nach dem Start schaltet es den Laser ein, bereitet das Display vor und liest einmal den aktuellen Zustand des Sensors ein. Danach läuft es in einer Dauerschleife: Immer wieder fragt es den Sensor ab, prüft den Taster und schaut, ob ein neuer Tag begonnen hat.

3.4.2 Zählen über den Signalwechsel

Würde das Programm einfach immer dann zählen, wenn der Strahl unterbrochen ist, käme es zu falschen Werten: Ein Brett, das langsam durch den Strahl läuft, würde mehrfach gezählt, solange es den Strahl blockiert. Deshalb wird nicht der Zustand selbst gezählt, sondern der Wechsel des Zustands.

Gezählt wird genau in dem Moment, in dem der Strahl von „frei“ auf „unterbrochen“ wechselt. Dafür merkt sich das Programm den vorherigen Zustand und vergleicht ihn mit dem aktuellen. Nur beim Übergang von frei zu unterbrochen wird der Zähler um eins erhöht. Solange das Brett den Strahl weiter blockiert, passiert nichts mehr. Erst wenn der Strahl wieder frei ist und das nächste Brett kommt, wird wieder gezählt.

Der Abschnitt der Schleife der das Zählen auslöst:

```
# Zählen: nur beim Wechsel auf "unterbrochen" (LOW), mit Entprellung
if status != letzter_status:
    if status == GPIO.LOW and (jetzt - letzter_impuls) >= ENTPRELLZEIT:
        letzter_impuls = jetzt
        brett_zahlen()
    letzter_status = status
```

3.4.3 Entprellung gegen Mehrfachzählung

Beim Umschalten kann das Signal kurz mehrmals hin- und herspringen, bevor es sich beruhigt. Ohne Gegenmaßnahme würde der Zähler dadurch bei einem einzigen Brett mehrfach hochzählen. Um das zu verhindern, wurde eine kurze Wartezeit von 50 Millisekunden eingebaut. Nach jeder Zählung ignoriert das Programm für diese kurze Zeit weitere Wechsel. Erst danach ist die nächste Zählung wieder möglich. Diese Wartezeit ist so kurz, dass selbst schnell aufeinanderfolgende Bretter noch sicher erkannt werden, aber lang genug, um das Flackern auszublenden.

3.4.4 Fehlzählung beim Start vermeiden

In einer frühen Fassung zählte das Programm direkt nach dem Start einmal zu viel. Der Grund: Beim Start stand noch nicht fest, in welchem Zustand sich der Sensor gerade befindet, und das Programm wertete das als ersten Wechsel. Behoben wurde das, indem der tatsächliche Zustand des Sensors einmal eingelesen wird, bevor die Schleife startet. Damit beginnt der Vergleich mit einem richtigen Wert, und die zusätzliche Zählung am Anfang fällt weg.

3.5 Anzeige, Chargen und Speicherung

3.5.1 Anzeige auf dem Display

Das Display hat zwei Zeilen. In der oberen Zeile steht die laufende Charge mit ihrem aktuellen Zählerstand, zum Beispiel „C2: 37“ für die zweite Charge mit 37 gezählten Brettern. In der unteren Zeile läuft die Uhrzeit mit. So sieht man auf einen Blick, wie viele Bretter in der aktuellen Charge schon gezählt wurden.

Anfangs wurde das Display vor jeder Aktualisierung komplett gelöscht und neu beschrieben. Das führte aber zu einem sichtbaren Flackern, weil die Anzeige bei jedem Brett kurz leer war. Gelöst wurde das mit einer eigenen Funktion, die gezielt nur die Zeile neu schreibt, die sich geändert hat, statt das ganze Display zu löschen. Dadurch ändern sich nur die wenigen Stellen, die sich wirklich ändern, und die Anzeige bleibt ruhig und gut lesbar. Das ist für eine Anlage, die den ganzen Tag läuft, deutlich angenehmer.

3.5.2 Chargen über das Taster-Modul

Damit man die Zählung sinnvoll aufteilen kann, gibt es das Konzept der Charge. Eine Charge ist eine zusammengehörige Menge gezählter Bretter, zum Beispiel ein Auftrag oder eine Lieferung. Über das Taster-Modul (KY-004) lässt sich jederzeit eine neue Charge starten. Ein Tastendruck schließt die laufende Charge ab, sichert deren Wert und beginnt eine frische Zählung. So lassen sich verschiedene Aufträge sauber voneinander trennen, ohne dass man die Anlage neu starten oder etwas am Programm ändern muss.

3.5.3 Speicherung in einer XML-Datei

Die Zähldaten werden dauerhaft in einer Datei gespeichert, damit sie auch nach einem Neustart noch da sind. Gewählt wurde dafür das Format XML, weil es übersichtlich aufgebaut ist und sich gut verschachteln lässt. Die einzelnen Chargen werden dabei dem jeweiligen Tag untergeordnet. Jeder Tag hat einen Gesamtwert, und jede Charge merkt sich ihre Nummer, die Start- und Endzeit, die Stückzahl und ob sie noch läuft oder schon abgeschlossen ist. So sieht man auf einen Blick, welche Chargen an welchem Tag gezählt wurden, und kann die Daten sowohl tageweise als auch über längere Zeiträume auswerten. Der grundsätzliche Aufbau sieht so aus:

```
<zaehler>
  <tag datum="2026-06-29" gesamt="79">
    <charge nummer="1" start="08:00:00" ende="09:30:00" wert="42" status="beendet" />
    <charge nummer="2" start="09:30:00" wert="37" status="offen" />
  </tag>
</zaehler>
```

Eine abgeschlossene Charge hat eine Endzeit und den Status „beendet“. Die gerade laufende Charge hat noch keine Endzeit und steht auf „offen“. Genau dieser Status ist wichtig für den Schutz vor Datenverlust, der im nächsten Abschnitt beschrieben wird.

3.5.4 Automatischer Tageswechsel um Mitternacht

Damit am Schichtende niemand etwas von Hand nachtragen muss, erkennt das Programm den Tageswechsel selbst. Um Mitternacht schließt es den abgelaufenen Tag ab, sichert dessen Werte und legt für den neuen Tag einen eigenen Eintrag an. Dadurch sind die Tage sauber voneinander getrennt, und die Zählung startet für den neuen Tag automatisch wieder bei null, ohne dass jemand eingreifen muss.

3.5.5 Schutz vor Datenverlust bei Stromausfall

Ein Stromausfall darf nicht dazu führen, dass die bis dahin gezählten Bretter verloren gehen. Im Moment des Ausfalls kann der Pi selbst nichts mehr speichern, weil dann kein Strom mehr da ist. Deshalb wird der Stand schon vorher laufend gesichert: Nach jedem gezählten Brett schreibt das Programm den aktuellen Gesamtwert und den Stand der laufenden Charge in die XML-Datei. So liegt der letzte Stand immer schon auf der Speicherkarte.

Damit auch die laufende Charge nicht verloren geht, wird sie in der Datei als „offen“ markiert. Startet das Programm nach einem Stromausfall neu, sucht es zuerst nach einer solchen offenen Charge. Findet es eine, zählt es genau dort weiter, wo es aufgehört hat, mit derselben Nummer und derselben Startzeit. Es geht also kein einziges Brett verloren, und es wird auch keine neue Charge aus Versehen begonnen.

Die richtige Uhrzeit kommt dabei von der Echtzeituhr (DS1307). Dadurch landet der wiederhergestellte Stand auch nach einem Stromausfall am richtigen Tag, selbst wenn der Pi kein Internet hat. Uhr und laufendes Speichern greifen also ineinander: Die Uhr sorgt für das richtige Datum, das häufige Speichern für den lückenlosen Zählerstand.

3.6 Bedienung der Anlage

Die Bedienung ist bewusst einfach gehalten, damit auch jemand ohne technisches Vorwissen damit zurechtkommt. Im Normalfall muss man gar nichts tun: Sobald der Pi mit Strom versorgt und das Programm gestartet wird, startet der Laser und das Display zeigt den Zählerstand. Danach läuft die Anlage einfach mit und zählt jedes Brett, das durch den Strahl läuft.

Soll eine neue Charge beginnen, zum Beispiel weil ein neuer Auftrag startet, genügt ein Druck auf den Taster. Die bisherige Charge wird gespeichert und die Zählung beginnt frisch. Beim Aufstellen ist nur darauf zu achten, dass der Laser genau auf das Lichtsensor-Modul zeigt.

4 Test und Qualitätssicherung

4.1 Vorgehen beim Testen

Getestet wurde nicht erst am Ende, sondern immer wieder zwischendurch, nach jedem größeren Schritt. So fielen Fehler früh auf und ließen sich gleich beheben, statt sich bis zum Schluss anzusammeln. Geprüft wurde sowohl der normale Betrieb als auch das Verhalten in schwierigen Fällen, zum Beispiel bei sehr langsamen oder sehr schnellen Brettern.

4.2 Testfälle und Ergebnisse

Die folgende Tabelle zeigt die wichtigsten Tests und ihr Ergebnis. Für die Tests wurden Gegenstände von Hand durch die Lichtschranke geführt und die Anzeige sowie die gespeicherten Daten kontrolliert.

Test	Vorgehen	Ergebnis
Zählgenauigkeit	50 Gegenstände durchgeführt und mit der Anzeige verglichen	bestanden
Keine Doppelzählung	ein Gegenstand sehr langsam durch den Strahl geführt	bestanden
Schneller Durchlauf	Gegenstand schnell hintereinander durchgeführt	bestanden
Strahl dauerhaft blockiert	Hand dauerhaft in den Strahl gehalten	bestanden
Verhalten beim Start	Programm bei freiem Strahl neu gestartet	bestanden
Anzeige im Dauerbetrieb	Display über längere Zeit beobachtet	bestanden
Neue Charge	Taster gedrückt und neue Zählung geprüft	bestanden
Stromausfall	Stecker bei laufender Charge gezogen und neu gestartet	bestanden
Datenspeicherung	XML-Datei nach mehreren Durchläufen geöffnet und geprüft	bestanden
Tageswechsel	Systemzeit auf kurz vor Mitternacht gestellt	bestanden

Tabelle 7: Testfälle und Ergebnisse

4.3 Besondere Prüfungen

Besonders genau wurde geprüft, dass kein Brett doppelt gezählt wird. Genau das war bei den ersten Sensoren das größte Risiko. Der Test mit einem bewusst sehr langsam durchgeführten Gegenstand hat bestätigt, dass jeder Gegenstand genau einmal gezählt wird, auch wenn es den Strahl länger blockiert. Ebenso wurde der umgekehrte Fall geprüft: Bleibt die Hand dauerhaft im Strahl, läuft der Zähler nicht weiter hoch.

Auch die Speicherung wurde geprüft. Die erzeugte XML-Datei wurde nach mehreren Durchläufen geöffnet und kontrolliert, ob die Werte vollständig und plausibel sind und ob die Chargen richtig dem jeweiligen Tag zugeordnet werden. Erst danach galt die Speicherung als sicher funktionierend.

Besonders wichtig war der Test für den Stromausfall. Dafür wurde mitten in einer laufenden Charge der Stecker gezogen und der Pi danach neu gestartet. Das Ergebnis war wie gewünscht: Der Zählerstand der Charge war noch da, und die Anlage zählte genau an dieser Stelle weiter, ohne dass ein Brett verloren ging oder eine neue Charge begonnen wurde.

5 Projektabschluss

5.1 Soll-Ist-Vergleich

Die Gesamtzeit von 40 Stunden wurde eingehalten. Innerhalb der Phasen gab es aber Verschiebungen, vor allem beim Hardware-Aufbau.

Projektphase	Soll (h)	Ist (h)	Differenz (h)
Analyse und Planung	5	4	-1
Anforderungen klären, Ist-Situation aufnehmen	2	2	-1
Schaltplan erstellen, Komponentenliste und Projektstruktur festlegen	2	2	0
Hardware-Aufbau	5	9	+4
Betriebssystem installiert, SSH-Zugang eingerichtet, Echtzeituhr konfiguriert, VS Code Remote verbunden	2	2	0
Sensorauswahl: mehrere Sensor-Varianten getestet und verworfen (Infrarot-Empfänger, Abstandssensor),	---	5	+5
Finalen Aufbau verdrahtet: Laser-/Lichtsensor-Modul, Taster-Modul und Display	3	2	-1
Programmierung	16	14	-2
GPIO-Eingang konfiguriert, Sensor-Signal ausgelesen	5	5	0
Zählerlogik entwickelt: Fehlzählung beim Start vermieden, Chargen-Funktion über Taster-Modul	5	4	-1
LCD-Ausgabe implementiert: Anzeige von Charge und Uhrzeit	3	2	-1
Speicherung als XML-Datei realisiert: automatischer Tageswechsel, Schutz vor Datenverlust bei Stromausfall	3	3	0
Test und Validierung	6	6	0
Funktionstest im Normalbetrieb: Zählgenauigkeit, Anzeige und Chargenwechsel geprüft	3	3	0
Fehlertests: Doppelzählung, Sensorblockade, Stromausfall und Tageswechsel simuliert	3	3	0
Dokumentation und Präsentation	8	7	-1
Technische Dokumentation erstellt: Schaltplan, Code-Kommentare	4	6	+2
Prototyp aufgebaut und Funktionsweise vorgeführt	4	2	-2
Gesamt	40	40	0

Tabelle 8: Soll-Ist-Vergleich

Der Mehraufwand beim Hardware-Aufbau (+4 Stunden) kam durch die Suche nach dem passenden Sensor, weil mehrere Varianten ausprobiert werden mussten. Diese Zeit wurde an anderen Stellen wieder eingespart: Die Programmierung ging dank klarer Struktur schneller als gedacht, und auch Planung und Dokumentation brauchten etwas weniger Zeit. Unterm Strich blieb es genau bei den geplanten 40 Stunden.

Auch bei den Kosten gab es keine Überraschungen. Alle Bauteile waren bereits vorhanden, und selbst bei einer Neuanschaffung würden der Raspberry Pi, die beiden Sensor-Module, der Taster und das Display zusammen nur wenige Euro kosten. Die Software ist vollständig kostenlos, da sowohl Python als auch die verwendeten Bibliotheken frei verfügbar sind.

5.2 Abweichungen zur Projektskizze

Im Lauf der Umsetzung gab es vier Punkte, an denen es anders lief als ursprünglich geplant:

1. **Anderer Sensor als geplant.** Geplant war eine fertige Infrarot-Lichtschanke. Diese lief im Test aber nicht zuverlässig, weil der Empfänger ein getaktetes Signal braucht und ein Dauerlicht ausblendet. Nach mehreren Versuchen fiel die Wahl auf ein Laser-Modul zusammen mit einem Lichtsensor-Modul. Diese Lösung läuft stabil und ist leichter auszurichten. Dadurch ist beim Hardware-Aufbau mehr Zeit angefallen.
2. **Speicherung als XML statt als CSV.** Ursprünglich sollten die Daten als einfache CSV-Datei gespeichert werden. Stattdessen wurde XML gewählt, weil sich damit die Chargen sauber unter dem jeweiligen Tag verschachteln lassen. Die Daten bleiben so übersichtlicher und sind leichter auszuwerten als eine flache Liste.
3. **Überwachung bei Sensorausfall noch nicht umgesetzt.** Eine zusätzliche Funktion, die einen Ausfall des Sensors erkennt und meldet, war als Erweiterung angedacht. Sie wurde im Rahmen der 40 Stunden nicht mehr umgesetzt, weil die Sensor-Suche mehr Zeit gebraucht hat. Die Funktion lässt sich später ohne zusätzliche Hardware ergänzen.
4. **Datensicherheit zusätzlich verbessert.** Über die ursprüngliche Planung hinaus wurde der Schutz vor Datenverlust ausgebaut. Eine Echtzeituhr (DS1307) sorgt für die richtige Zeit auch ohne Internet, und der Stand wird nach jedem Brett gespeichert. Nach einem Stromausfall läuft die offene Charge dort weiter, wo sie war. Damit geht selbst bei einem plötzlichen Stromausfall kein Brett verloren.

Alle Abweichungen wurden festgehalten. Auf den geplanten Zeit- und Kostenrahmen hatten sie keine negativen Auswirkungen, und die XML-Lösung sowie der zusätzliche Schutz vor Datenverlust sind gegenüber der ursprünglichen Planung sogar ein Vorteil.

6 Fazit und Ausblick

Das Projekt wurde erfolgreich und innerhalb der geplanten 40 Stunden abgeschlossen. Am Ende steht ein System, das die Bretter ganz von allein zählt, ohne dass es sie berühren muss, den Stand jederzeit auf dem Display anzeigt und die Zahlen in Chargen sauber speichert. Der Tageswechsel passiert automatisch. Damit ersetzt das System das bisherige Zählen von Hand und liefert eine genaue Grundlage für spätere Auswertungen.

Der größte AHA-Effekt lag bei der Hardware. Erst durch das Ausprobieren mehrerer Sensoren wurde klar, worauf es bei einer Lichtschranke ankommt und warum nicht jeder Sensor dafür geeignet ist. Auf der Software-Seite waren das Zählen über den Signalwechsel, die Entprellung und die flackerfreie Anzeige die wichtigsten Punkte. Diese Erfahrungen lassen sich gut auf ähnliche Aufgaben übertragen.

Für die Zukunft gibt es mehrere Möglichkeiten, das System zu erweitern:

- Eine Überwachung ergänzen, die einen Sensorausfall erkennt und meldet.
- Die Daten in einer richtigen Datenbank statt in einer XML-Datei speichern, wenn größere Mengen anfallen.
- Eine einfache Webseite bauen, die den Zählerstand im Browser anzeigt.
- Mehrere Sensoren gleichzeitig betreiben und das System auf andere Bereiche im Sägewerk übertragen.